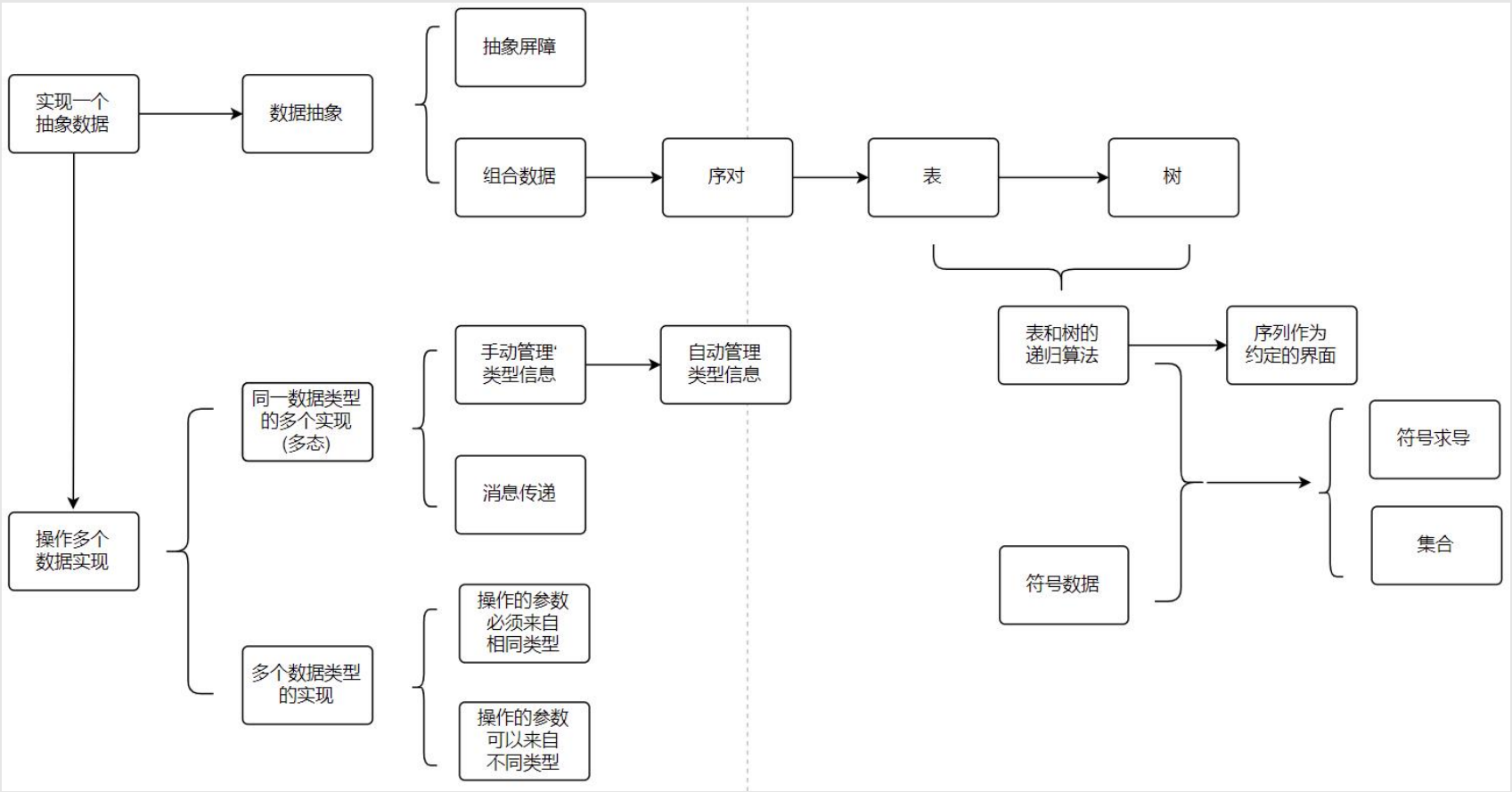
计算机科学导论-SICP
第二章 构造过程抽象
2.3小节 符号数据

王超

Center for Research and Innovation in Software Engineering (RISE), Southwest University

2022 年 11 月 11 日

第二章的学习思路


实现一个
抽象数据
数据抽象
抽象屏障
组合数据
序对
表
树
表和树的
递归算法
序列作为
约定的界面
符号求导
集合
符号数据
操作多个
数据实现
同一数据类型
的多个实现
(多态)
手动管理
类型信息
自动管理
类型信息
消息传递
多个数据类型
的实现
操作的参数
必须来自
相同类型
操作的参数
可以来自
不同类型

2.3节讲了什么故事

- 使用符号数据
- 更多表和树的应用例子

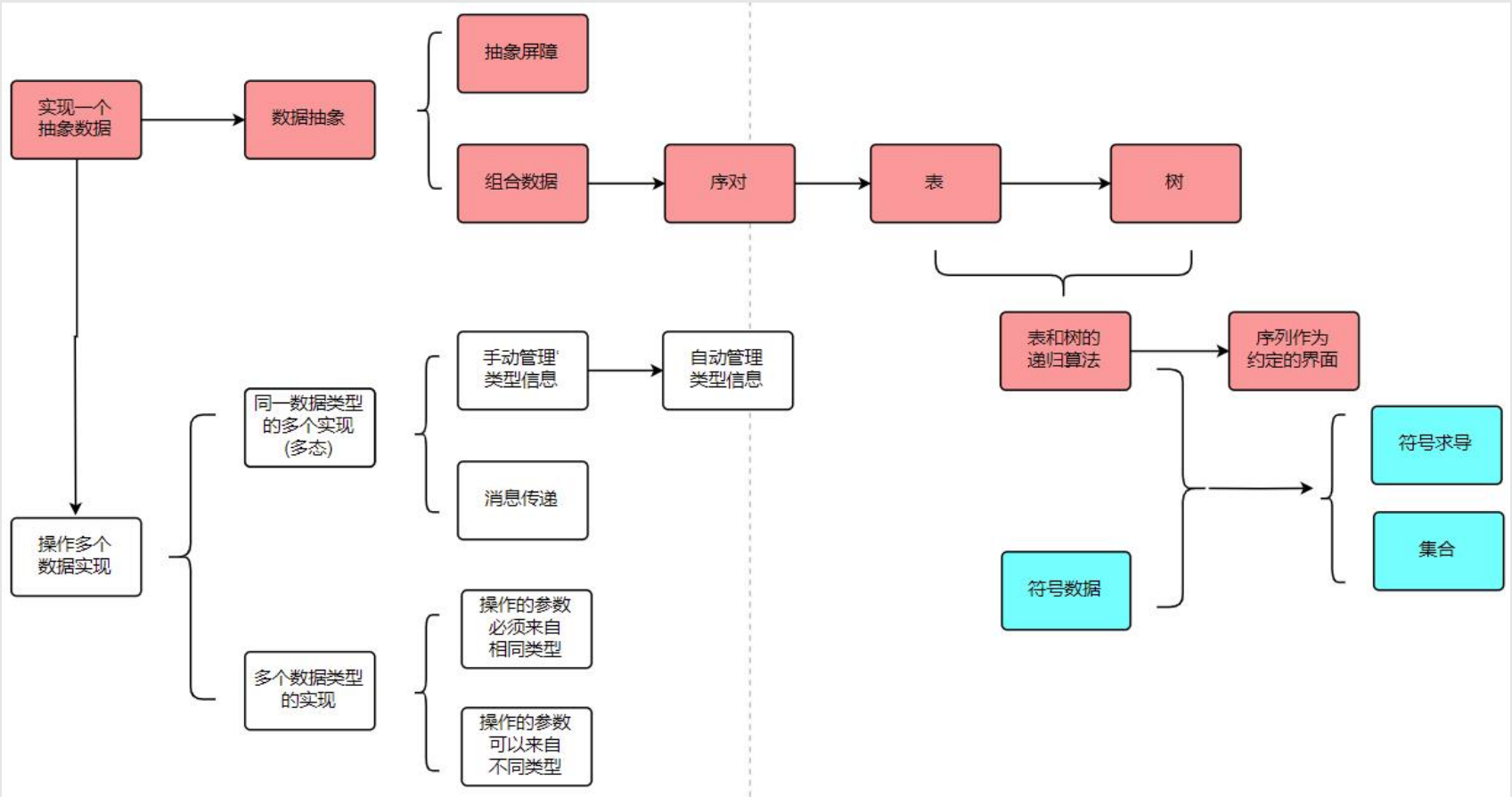
2.3节的PPT涉及哪些内容

- 符号数据
- 实例：符号求导
- 实例：集合的表示

内容调整

- 2.3.4节不讲

2.3的内容


实现一个
抽象数据
数据抽象
抽象屏障
组合数据
序对
表
树
表和树的
递归算法
序列作为
约定的界面
符号求导
集合
符号数据
操作多个
数据实现
同一数据类型
的多个实现
(多态)
手动管理
类型信息
自动管理
类型信息
消息传递
多个数据类型
的实现
操作的参数
必须来自
相同类型
操作的参数
可以来自
不同类型

大纲

- 使用符号数据
- 实例：符号求导
- 实例：集合的表示
- 结束

’操作

- 很多程序需要显示和处理符号
- 编程语言需提供字符操作

’操作

- Scheme提供
- ’+字符串，或’+(其他内容)
- 求值时，将’后面的内容视为文字而不是变量
- 求值结果是字符串

’+字符串

- 求值结果是后面紧接着的文字，直到遇到空格为止
- 例如，’swu的求值结果是字符串swu。而不使用’时swu被认为是一个变量，其求值结果是变量swu的值
- 类比：’太平洋 的中间是平，而 太平洋 的中间需要先求变量太平洋所代表的值，再求这个值的中间

’+(s)

- 求值结果是(+ 以文字形式出现的s +)
- 例如，’(a 1 2)的求值结果是表(a 1 2)
- 注意，这里不会代入a的值
- 注意，求值结果(a 1 2)是一个表，(car ’(a 1 2))=a，(cadr ’(a 1 2))=1，(caddr ’(a 1 2))=2

’可以对表的部分元素使用

- 当变量a和c的值分别为1和2时，(list a ’b c)的求值结果是(1 b 2)

判断字符串的相同

(eq? a b)

- Scheme提供的过程
- a和b都是字符串
- 判断a和b是否内容相同

(memq item x)

- 判断字符串item是否是表x的元素
- 如果是，则返回表自item第一次出现时余下的部分。否则，返回false
- 例如，(memq 'abc '(a b abc d e))=(abc d e)
- 注意，memq的求值结果并不是同一“类型”。这样做的用处是，只要其求值结果不是false，就一定是一个表

```
(define (memq item x)
  (cond ( (null? x) false)
        ( (eq? item (car x) ) x)
        ( else (memq item (cdr x))))))
```

eq?的局限性

- eq?判断字符串的相等，但无法判断字符串表的相等
- 在后续章节中会引入equal?过程
- 有时会有奇怪的结果
- 在后续章节我们会再讨论判断“相同”这个问题

```
Welcome to DrRacket, version 8.2 [cs].
Language: sicp, with debugging; memory limit: 128 MB.
> (define a '(1 2 3))
> (define b '(1 2 3))
> (eq? a b)
#f
> (equal? a b)
#t
```

大纲

- 使用符号数据
- 实例：符号求导
- 实例：集合的表示
- 结束

例子：符号求导

- 执行代数表达式的符号求导
- 输入是文本形式的代数表达式和变量
- 代数表达式允许常数，变量，以及加法和乘法运算
- 输出是文本形式的求导结果
- 注意，第一章我们已经实现了数值求导的计算，这里实现的是字符形式的代数表达式的求导

符号+按愿望思维

(deriv expr1 var)

- 输入是一个字符串形式的代数表达式expr1和字符串形式的变量var
- 输出是一个字符串形式的代数表达式expr2，且expr2是expr1对var求导后的结果
- 例子：(deriv '(+ (* 2 (* x x)) (* x 3)) 'x)的求值结果应该是(+ (* 2 x) 3)，即$2 * x^2 + 3 * x$对x求导结果为2*x+3
- 输入和输出都是字符串，写成前缀表达式的形式

求导规则

$$\frac{dc}{dx} = 0 \quad c\ is\ a\ constant\ or\ non-x\ variable$$

$$\frac{dx}{dx} = 1$$

$$\frac{d(u+v)}{dx} = \frac{du}{dx} + \frac{dv}{dx}$$

$$\frac{d(uv)}{dx} = u\frac{dv}{dx} + v\frac{du}{dx}$$

基于求导规则设计算法

- 使用按愿望思维
- 需要提供如下操作

针对第一个式子和第二个式子

- (number? exp)：判断a的值是否是数字。由Scheme提供
- (variable? exp)：判断表达式exp是否是单个变量
- (same-variable? v1 v2)：判断变量v1和v2是否是同一个变量

针对第三个式子

- (sum? exp)：判断表达式exp是否是最外层运算为加法的表达式
- (addend exp)：返回exp的被加数，exp为最外层为加法的表达式
- (augend exp)：返回exp的加数，exp为最外层为加法的表达式
- (make-sum a b)：生成被加数为a，加数为b的表达式

针对第四个式子

- (product? exp)：判断表达式exp是否是最外层运算为乘法的表达式
- (multiplier exp)：返回exp的被乘数，exp为最外层为乘法的表达式
- (multiplicand exp)：返回exp的乘数，exp为最外层为乘法的表达式
- (make-product a b)：生成被乘数为a，乘数为b的表达式

根据求导公式和这些过程，写出求导算法

```
;求代数表达式的导数
(define (deriv exp var)
  (cond ( (number? exp) 0)
        ( (variable? exp)
          (if (same-variable? exp var) 1 0))
        ( (sum? exp)
          (make-sum (deriv (addend exp) var)
                    (deriv (augend exp) var)))
        ( (product? exp)
          (make-sum (make-product (multiplier exp)
                                  (deriv (multiplicand exp) var))
                    (make-product (deriv (multiplier exp) var)
                                  (multiplicand exp))))
        (else error "error in deriv")))
```

实现和变量相关的过程

- 一个变量变量被实现为一个字符串
- Scheme提供过程(symbol? x)判断x是否是一个字符串
- (varialbe? x)：使用(symbol? x)
- (same-variable? a b)：确认a和b都是变量，并使用eq?判断是否是同一变量

```
;变量相关的操作
(define (variable? x)(symbol? x))

(define (same-variable? a b)
  (and (variable? a) (variable? b) (eq? a b)))
```

实现加法相关的过程

- 表达式exp1+exp2被实现为表('+ exp1 exp2)
- 注意，表的首元素不是过程+，而是符号'+
- 如何判断表达式x是否是加法表达式：判断(car x)是否为'+
- 如何求加法表达式s的被加数：(cadr s)
- 如何求加法表达式s的加数：(caddr s)
- 由于表中有三个元素，因此使用car、cadr和caddr

```
;加法相关的操作
(define (sum? x)
  (and (pair? x) (eq? (car x) '+)))

(define (make-sum a b) (list '+ a b))

(define (addend s) (cadr s))

(define (augend s) (caddr s))
```

实现乘法相关的过程

- 表达式exp1*exp2被实现为表('* exp1 exp2)
- 注意，表的首元素不是过程*，而是符号'*
- 如何判断表达式x是否是乘法表达式：判断(car x)是否为'*
- 如何求乘法表达式p的被乘数：(cadr p)
- 如何求乘法表达式p的乘数：(caddr p)
- 由于表中有三个元素，因此使用car、cadr和caddr

```
(define (make-product a b) (list '* a b))

(define (product? x)
    (and (pair? x) (eq? (car x) '*)))

(define (multiplier p) (cadr p))

(define (multiplicand p) (caddr p))
```

抽象屏障

- 代数表达式的编写过程同样展示了抽象屏障



化简求导结果

- 求导结果完全正确，但包含大量冗余计算

```
(deriv '(+ x 3) 'x)
(deriv '(* x y) 'x)
(deriv '(* (* x y) (+ x 3)) 'x)
(deriv '(* x (* x x)) 'x)
(deriv '(* x 3) 'x)
```

```
欢迎使用 DrRacket, 版本 8.1 [cs].
语言: sicp，带调试; memory limit: 128 MB.
(+ 1 0)
(+ (* x 0) (* 1 y))
(+ (* (* x y) (+ 1 0)) (* (+ (* x 0) (* 1 y)) (+ x 3)))
(+ (* x (+ (* x 1) (* 1 x))) (* 1 (* x x)))
(+ (* x 0) (* 1 3))
>
```

- x+3对x求导的结果是1+0，因为x和3分别对x的求导结果是1和0
- x*y对x求导的结果是x*0+1*y
- (x*y)*(x+3)对x的求导结果是((x*y)*(1+0))+((x*0+1*y) * (x+3))
- (x*(x*x))对x的求导结果是((x* (1*x+1*x)) + 1*(x*x))
- (* x 3)对x的求导结果是x*0+1*3

这些结果其实可以被大大的化简

- x+3对x求导的结果是1+0 = 1
- x*y对x求导的结果是x*0+1*y = y
- (x*y)*(x+3)对x的求导结果是((x*y)*(1+0))+((x*0+1*y) * (x+3)) = 2*x*y+3*y
- (x*(x*x))对x的求导结果是((x* (1*x+1*x)) + 1*(x*x)) = 3*x*x
- (* x 3)对x的求导结果是x*0+1*3 = 3

如何去除冗余，化简结果

求导过程中明显的冗余计算

- 1+0这样的纯数字计算应该被化简，对乘法亦然
- x*0和1*y中的乘以0和乘以1应该被化简。x*0+1*y的化简结果是0+y，进一步化简为y

这两种情况

- 把一个较长的加法或乘法表达式化简
- 可以在构造加法和乘法表达式时执行

化简加法

- a和0的和、0和a的和显然是a
- 两个数字的和显然是一个数字
- 对其他情况，像原来一样计算
- (=number? exp n)用来判断exp是一个数字且值为n

```
(define (make-sum a b)
  (cond ( (=number? a 0) b)
        ( (=number? b 0) a)
        ( (and (number? a) (number? b))
          (+ a b))
        (else (list '+ a b))))


(define (=number? exp n)
  (and (number? exp) (= exp n)))
```

化简乘法

- a和0的积、0和a的积显然是0
- a和1的积、1和a的积显然是a
- 两个数字的积显然是一个数字
- 对其他情况，像原来一样计算

```
(define (make-product a b)
  (cond ( (or (=number? a 0) (=number? b 0)) 0)
        ( (=number? b 1) a)
        ( (=number? a 1) b)
        ( (and (number? a) (number? b))
          (* a b))
        ( else (list '* a b))))
```

```
;例子
(deriv '(+ x 3) 'x)
(deriv '(* x y) 'x)
(deriv '(* (* x y) (+ x 3)) 'x)
(deriv '(* x (* x x)) 'x)
(deriv '(* x 3) 'x)
```

```
欢迎使用 DrRacket, 版本 8.1 [cs].
语言: sicp，带调试; memory limit: 128 MB.
1
y
(+ (* x y) (* y (+ x 3)))
(+ (* x (+ x x)) (* x x))
3
>
```

新算法的求导结果

- 1+0 ⇒ 1
- x*0+1*y ⇒ y
- ((x*y)*(1+0))+((x*0+1*y) * (x+3)) ⇒ x*y+y*(x+3)
- ((x* (1*x+1*x)) + 1*(x*x)) ⇒ x*(x+x)+x*x
- x*0+1*3 ⇒ 3

修改后的求导过程仍有修改空间

- x*(x+x)+x*x可以进一步简化为3*x*x

大纲

- 使用符号数据
- 实例：符号求导
- 实例：集合的表示
- 结束

集合

- 一个集合包含若干元素，可以是有穷集或无穷集
- 本小节实现有穷集合

集合作为抽象数据

- 提供给用户如下接口
- (union-set set1 set2)：计算两个集合set1和set2的并集
- (intersection-set set1 set2)：两个集合set1和set2的交集
- (element-of-set? x set)：判断元素x是否属于集合set
- (adjoin-set x set)：计算把元素x放入集合set后的新集合
- 常量empty-set，代表空集

三种实现方法

- 使用未排序的表实现集合
- 使用排序的表实现集合
- 使用二叉树实现集合

时间增长阶越来越好

方法一：使用未排序的表实现集合

- 一个集合实现为一个未排序的表，每个元素出现一次
- 例如，集合$\{a_1, \ldots, a_k\}$实现为表$(a_1 \ \ldots \ a_k)$
- empty-set实现为nil
- (element-of-set? x set)：递归地遍历表，如果遇到x则返回true；如果表遍历完仍未遇到x，则返回false
- (adjoin-set x set)：如果元素x已在集合set中，则求值结果为set；否则，求值结果为将x放入表set中得到的表
- element-of-set?中使用equal?，说明集合元素除了数字和字符，还可以是表

```
;使用未排序的表实现集合
(define empty-set nil)

(define (element-of-set? x set)
  (cond ( (null? set) false)
        ( (equal? x (car set)) true)
        ( else (element-of-set? x (cdr set)))))

(define (adjoin-set x set)
  (if (element-of-set? x set)
      set
      (cons x set)))
```

(intersection-set set1 set2)

- 当set1的首元素在set2中时：(intersection-set set1 set2)=(cons (car set1) (intersection-set (cdr) set2))
- 当set1的首元素不在set2中时：(intersection-set set1 set2)=(intersection-set (cdr set1) set2)
- 当扫描完set1的所有元素后，使用递归保存的con，将需要保存的信息组合为一张表

```
(define (intersection-set set1 set2)
  (cond ( (or (null? set1) (null? set2)) nil)
        ( (element-of-set? (car set1) set2)
          (cons (car set1)
                (intersection-set (cdr set1) set2)))
        (else (intersection-set (cdr set1) set2))))
```

教材练习 2.56

- (union-set set1 set2)：合并两个表的元素，类似append
- 注意，一个集合被实现为一张表，表中不允许有相同元素
- 当set1的首元素在set2中时：(union-set set1 set2)=(union (cdr set1) set2)
- 当set1的首元素不在set2中时：(union-set set1 set2)=(cons (car set1) (union-set (cdr) set2))

```
(define (union-set set1 set2)
  (cond ( (null? set1) set2)
        ( (null? set2) set1)
        ( (element-of-set? (car set1) set2)
          (union-set (cdr set1) set2))
        (else (cons (car set1)
                    (union-set (cdr set1) set2)))))
```

```
(element-of-set? 'a '(1 2 a b 3))
(adjoin-set 'b '(10 a 20))
(intersection-set '(1 a 2 b 3 c) '(b 1 4 8))
(union-set '(1 a 2 b 3 c) '(b 1 4 8))
(adjoin-set 1 (adjoin-set 'a empty-set))
```

```
欢迎使用 DrRacket, 版本 8.1 [cs].
语言: sicp，带调试; memory limit: 128 MB.
#t
(b 10 a 20)
(1 b)
(a 2 3 c b 1 4 8)
(1 a)
>
```

- 如果把我们的集合操作视为一种集合语言
- 那么，最好用(adjoin-set 1 (adjoin-set 'a empty-set))，而不是'(1 a)，来表示集合$\{1, a\}$

- (element-of-set? x set)在最坏情况下需读整个表，因此有着线性的时间增长阶
- adjoin-set调用一次element-of-set?和一次cons，因此也有着线性的时间增长阶
- (intersection-set set1 set2)需要对表set1的每个元素使用一次element-of-set?。当set1和set2都包含n个元素时，在最坏情况下有着n^2的时间增长阶
- (union-set set1 set2)需要对表set1的每个元素使用一次element-of-set?。当set1和set2都包含n个元素时，在最坏情况下有着n^2的时间增长阶

方法二：使用排序的表实现集合

- 一个集合实现为一个已排序的表，元素从小到大出现，每个元素出现一次
- 为了让集合元素都可以比较，不妨假定集合元素都是数字
- 在未来，可以通过加入人为定义的序关系，让集合元素不仅是数字

empty-set不变

```
(define empty-set nil)
```

(element-of-set? x set)

- 以(element-of-set? 2 (list 3 4 5))为例，因为2 $<$ 3，且3是表中最小的元素，所以2不可能在表(3 4 5)中
- 改进：无需读完整张表
- 从头开始扫描set，如果读到x，则返回true
- 如果先读到若干比x小的元素，又读到比x大的某个y，则y之后的元素肯定仍大于x，此时返回false
- 在最好的情况下，可以在搜索的第一时间发现x不在set中
- 最坏情况下需要读完整张表，时间增长阶不变

```
(define (element-of-set? x set)
  (cond ( (null? set) false)
        ( (= x (car set)) true)
        ( (< x (car set)) false)
        ( else (element-of-set? x (cdr set)))))
```

(intersection-set set1 set2)

- 令人震惊的是，当使用排序的表实现集合时，intersection-set有一个线性时间的算法
- 之前的intersection-set多次使用element-of-set?，重复遍历表元素
- 这次的intersection-set的加速来自无需重复遍历表元素
- 局部性：如果数a的值小于排序表s的首元素，则a的值小于s中的每个元素，进而a不在s中
- 令x1和x2分别是set1和set2的首元素
- 若$x1 < x2$：x_1小于set2中任意元素，故$x1$不能放入交集
- 若$x1 = x2$：x1可以放入交集
- 若$x1 > x2$：x2小于set1中任意元素，故x2不能放入交集
- 当set1或set2为空时，结果为nil
- 每次比较之后，至少一个元素就不会被再处理。最多把两个表都几乎遍历完，故时间增长阶为线性

```
(define (intersection-set set1 set2)
  (if (or (null? set1) (null? set2))
      nil
      (let ( (x1 (car set1)) (x2 (car set2)) )
        (cond ( (= x1 x2)
                (cons x1 (intersection-set (cdr set1) (cdr set2))))
              ( (< x1 x2)
                (intersection-set (cdr set1) set2))
              ( (< x2 x1)
                (intersection-set set1 (cdr set2)))))))
```

例子：(intersection-set (2 6 8) (2 4 8))

- (intersection-set (2 6 8) (2 4 8))
- = (cons 2 (intersection-set (6 8) (4 8)))
- = (cons 2 (intersection-set (6 8) (8)))
- = (cons 2 (intersection-set (8) (8)))
- = (cons 2 (cons 8 (intersection-set nil nil)))
- = (cons 2 (cons 8 nil)) = (2 8)

教材练习2.62：实现(union-set set1 set2)

- 使用类似intersection-set的设计思路
- 局部性：如果数a的值小于排序表s的首元素，则a的值小于s中的每个元素，进而a不在s中
- 令x1和x2分别是set1和set2的首元素
- 若$x1 < x2$：x_1小于set2中任意元素，故$x1$放入并集
- 若$x1 = x2$：x1和x2只有一个放入并集
- 若$x1 > x2$：x2小于set1中任意元素，故x2放入并集
- 当set1为空时，结果为set2。当set2为空时，结果为set1
- 每次比较之后，至少一个元素就不会被再处理。最多把两个表都几乎遍历完，故时间增长阶为线性

```
(define (union-set set1 set2)
  (cond ( (null? set1) set2)
        ( (null? set2) set1)
        ((let ( (x1 (car set1)) (x2 (car set2)) )
           (cond ( (= x1 x2)
                   (cons x1 (union-set (cdr set1) (cdr set2))))
                 ( (< x1 x2)
                   (cons x1 (union-set (cdr set1) set2)))
                 ( (< x2 x1)
                   (cons x2 (union-set set1 (cdr set2)))))))))
```

例子：(union-set (2 6 8) (2 4 8))

- (union-set (2 6 8) (2 4 8))
- = (cons 2 (union-set (6 8) (4 8)))
- = (cons 2 (cons 4 (union-set (6 8) (8))))
- = (cons 2 (cons 4 (cons 6 (union-set (8) (8)))))
- = (cons 2 (cons 4 (cons 6 (cons 8 (union-set nil nil)))))
- = (cons 2 (cons 4 (cons 6 (cons 8 nil))))
- = (2 4 6 8)

教材练习2.61：实现(adjoin-set x set)

- 需要保证结果仍然是排序的表
- 若$x < (car\ set)$：(cons x set)
- 若$x = (car\ set)$：set
- 若$x > (car\ set)$：(cons (car set) (adjoin-set x (cdr set)))
- 当set为空时，结果为(list x)
- 最坏情况下遍历set，故时间增长阶为线性

```
(define (adjoin-set x set)
  (cond ( (null? set) (list x))
        ( (< x (car set))
          (cons x set))
        ( (= x (car set))
          set)
        ( (> x (car set))
          (cons (car set) (adjoin-set x (cdr set))))))
```

课堂思考题：给出(adjoin-set 16 (list 2 10))的执行过程

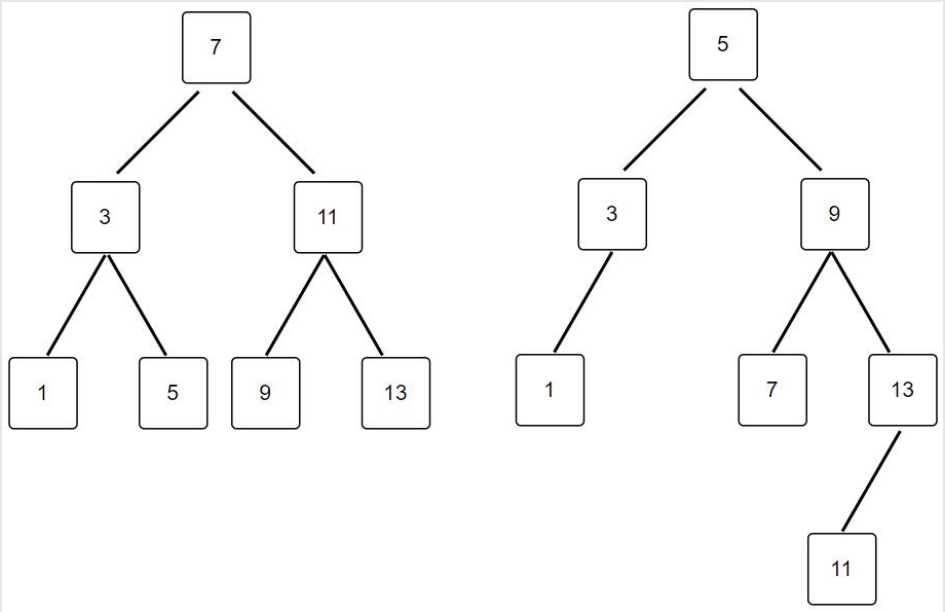
- intersection-set和union-set的时间增长阶较为理想
- 方法三将进一步改进element-of-set?和adjoin-set

方法三：使用二叉树实现集合

- 在2.2中我们讲了二分的思想
- 如果递归的集合操作在每次递归时能够去掉一半元素，就得到了对数时间增长阶的算法
- 把集合实现为某种二叉树，即每个节点有两棵子树的树
- 如果左右子树的叶节点数目相近，并且每次递归可以忽略左子树或右子树，则递归的时间增长阶为对数

具体实现：以集合(list 1 3 5 7 9 11 13)为例

- 树的每个节点有一个数值，而不是只有叶节点有数值
- 每个节点的数值都大于其左子树中每个节点的数值
- 每个节点的数值都小于其右子树中每个节点的数值



- 相同的内容可以由不同的树来存储

树的节点

- 一棵树被存储为一个表(entry left-branch right-branch)
- 分别是当前树根节点的数据，左子树和右子树
- (entry tree)：返回树tree的根节点的数据
- (left-branch tree)：返回树tree的左子树
- (right-branch tree)：返回树tree的右子树
- (make-tree entry left right)：生成树，根节点数值为entry，左右子树分别为left和right

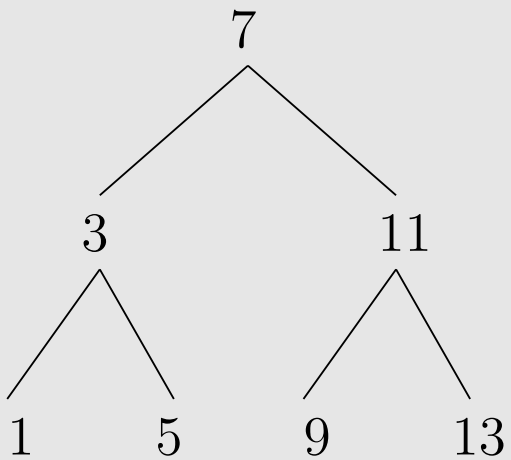
```
(define (enrty tree) (car tree))

(define (left-branch tree) (cadr tree))

(define (right-branch tree) (caddr tree))

(define (make-tree entry left right)
  (list entry left right))
```

- 当一棵树仅包含一个节点（叶节点）且数据为a时，表示为(a nil nil)
- 例如，下图中叶节点1和5表示为(1 nil nil)和(5 nil nil)，因此以3为根的子树表示为(3 (1 nil nil) (5 nil nil))
- 整棵树表示为(7 (3 (1 nil nil) (5 nil nil)) (11 (9 nil nil) (13 nil nil)))



(element-of-set? x set)

- 若set=nil，结果为false
- 令集合set对应的树其根节点的值为y
- $x = y$：x在集合set中，结果为true
- $x < y$：如果set中包含x，则这样的节点一定在y的左子树中。递归调用(element-of-set? x set的左子树)
- $x > y$：如果set中包含x，则这样的节点一定在y的右子树中。递归调用(element-of-set? x set的右子树)

```
(define (element-of-set? x set)
  (cond ( (null? set) false )
        ( (= x (entry set)) true)
        ( (< x (entry set))
          (element-of-set? x (left-branch set)))
        ( (> x (entry set))
          (element-of-set? x (right-branch set)))))
```

当树中每个点的左右子树大小相近时，算法的时间复杂度为对数

- 时间增长阶得到了改进

课堂思考题：说明在上一页的树中查询数字4和5的执行过程

(adjoin-set x set)

- 注意，该过程应该返回一棵二叉树，这棵二叉树的节点来自set和x，且一个数字出现一次
- 令集合set对应的树的根节点的值为y
- $x = y$：x在集合set中，计算结果正是set
- $x < y$：如果set中包含x，则这样的节点一定在y的左子树中
- 因此，结果为二叉树(y (adjoin-set x set的左子树) set的右子树)
- $x > y$：如果set中包含x，则这样的节点一定在y的右子树中
- 因此，结果为二叉树(y set的左子树 (adjoin-set x set的右子树))
- 如果set为空集，则结果为单节点树(list x nil nil)

```
(define (adjoin-set x set)
  (cond ( (null? set) (make-tree x nil nil) )
        ( (= x (entry set)) set)
        ( (< x (entry set))
          (make-tree (entry set)
                     (adjoin-set x (left-branch set))
                     (right-branch set)))
        ( (> x (entry set))
          (make-tree (entry set)
                     (left-branch set)
                     (adjoin-set x (right-branch set))))))
```

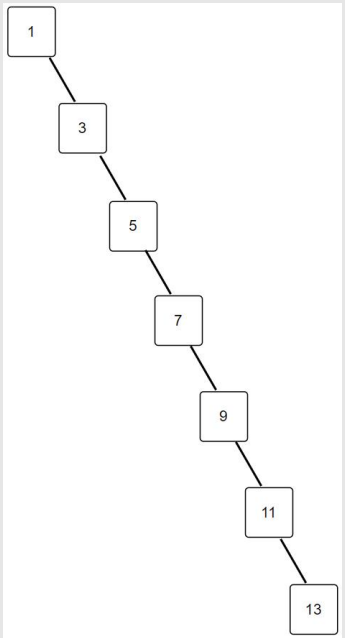
当树中每个点的左右子树大小相近时，算法的时间复杂度为对数

- 时间增长阶得到了改进

课堂思考题：针对前几页的树（称为set），说明(adjoin 6 set)的执行过程

保持树结构的平衡

- element-of-set?和adjoin-set对数算法的前提是树的结构足够平衡
- 但这一点并不总成立
- 例如，如果我们从empty-set出发，先后通过adjoin-set添加数字1、2、3、4、5、6、7，则得到如下树
- 这棵树的结构从树退化为线。在这样的树上执行element-of-set?和adjoin-set的时间增长阶为线性而不是对数
- 课堂思考题：解释这一点



- 不能寄希望于这样的事情不发生，而是要主动解决这个问题
- 引入make tree balance again的操作

操作分为两部分

- 第一部分：树 $\Rightarrow$ 有序表
- 第二部分：有序表 $\Rightarrow$ 树
- 这两部分都是线性的时间增长阶

教材练习2.63：(tree->list-2 tree)

- 将树tree转化为有序表
- 定义内部过程(copy-to-list tree2 result-list)
- 待转化结果为tree2，已转化结果为result-tree，且在最终结果中tree2在result-list之前
- 是教材练习2.63两个算法中的第二个

```
;把树转化为表，时间增长阶为线性
(define (tree->list-2 tree)
  ;当前已转化成的表为result-list，还待转化的树为tree2，且tree2一定在result-list之前
  (define (copy-to-list tree2 result-list)
    (if (null? tree2)
        result-list
        (copy-to-list (left-branch tree2)
                      (cons (entry tree2)
                            (copy-to-list (right-branch tree2)
                                          result-list)))))
  (copy-to-list tree nil))
```

例1：处理单节点树

- (copy-to-list (1 nil nil) nil)
- = (copy-to-list nil (cons 1 (copy-to-list nil nil)))
- = (copy-to-list nil (cons 1 nil))
- = (1)

例2：处理树(3 (1 nil nil) (5 nil nil))

- (copy-to-list (3 (1 nil nil) (5 nil nil)) nil)
- = (copy-to-list (1 nil nil) (cons 3 (copy-to-list (5 nil nil) nil)))，由已知
- = (copy-to-list (1 nil nil) (cons 3 (5)))
- = (copy-to-list (1 nil nil) (3 5))
- = (copy-to-list nil (cons 1 (copy-to-list nil (3 5))))
- = (copy-to-list nil (1 3 5))
- = (1 3 5)

课堂思考题：解释为何算法的时间增长阶为线性

教材练习2.64：(list− >tree elements)

- 将树tree转化为有序表
- 由于课时和算法篇幅原因，对这个算法，能叙述其工作原理即可，不要求写出代码

```
(define (list->tree elements)
  (car (partial-tree elements (length elements))))

;elts是有序表，n是数字，返回序对，其首元素为elts的前n个元素构成的平衡树，其次元素为余下的元素构成的有序对
(define (partial-tree elts n)
  (if (= n 0)
      (cons nil elts)
      (let ( (left-size (quotient (- n 1) 2)))
        (let ((left-result (partial-tree elts left-size)))
          (let ( ( left-tree (car left-result))
                 ( non-left-elts (cdr left-result))
                 ( right-size (- n (+ left-size 1))))
            (let ( (this-entry (car non-left-elts))
                   (right-result (partial-tree (cdr non-left-elts)
                                               right-size)))
              (let ( (right-tree (car right-result))
                     (remaining-elts (cdr right-result)))
                (cons (make-tree this-entry left-tree right-tree)
                      remaining-elts)))))))))
```

- 感兴趣的同学可尝试解释(partial-tree (1 3 5) 3)的执行过程
- 左子树和右子树的节点数目是直接算出的，进而递归算出左右子树
- 算法的最终部分仅是一步make-tree
- 大量的let代码使得无需重复计算

集合的交和并

教材练习2.65

- (intersection-set set1 set2)：先将set1和set2转为有序表list1和list2，然后将list1和list2视为两个以有序表形式实现的集合，进行集合的求交，最后将得到的有序表转为平衡二叉树
- (union-set set1 set2)：先将set1和set2转为有序表list1和list2，然后将list1和list2视为两个以有序表形式实现的集合，进行集合的求并，最后将得到的有序表转为平衡二叉树

```
(define (intersection-set set1 set2)
  (let ( (list1 (tree->list-2 set1))
         (list2 (tree->list-2 set2)))
    (list->tree (intersection-set-as-ordered-list list1 list2))))

(define (union-set set1 set2)
  (let ( (list1 (tree->list-2 set1))
         (list2 (tree->list-2 set2)))
    (list->tree (union-set-as-ordered-list list1 list2))))
```

- intersection-set-as-ordered-list是以有序表实现集合时的求交集
- union-set-as-ordered-list是以有序表实现集合时的求并集
- 使用let可以把对应的值固定下来，无需对同一个值求值多次

课堂思考题：解释为何intersection-set和union-set的时间复杂度为线性

大纲

- 使用符号数据
- 实例：符号求导
- 实例：集合的表示
- 结束

课堂总结

- 符号数据
- 递归+按愿望思维+文字
- 通过保持表的排序带来加速
- 另一种树的格式
- 通过保持树结构的平衡来加速
- 树 $\Rightarrow$ 排序表，排序表 $\Rightarrow$ 树

Scheme语法

- ’
- eq?, number?, symbol?

学习的难点不在于Scheme的语法，而在于编程思想

本节内容与其他知识的联系

在后续章节中会进一步进展的

- 2.3.4节给出了另一种树的格式

在数据结构课程中，将讲解红黑树等平衡树算法

课后题精选-2.3节

练习级

- 针对使用未排序表实现集合，实现过程(set-minus set1 set2)，执行结果为这样一个集合，其元素在集合set1中，而不在集合set2中
- 针对使用排序表实现集合，实现过程(set-minus set1 set2)，执行结果为这样一个集合，其元素在集合set1中，而不在集合set2中

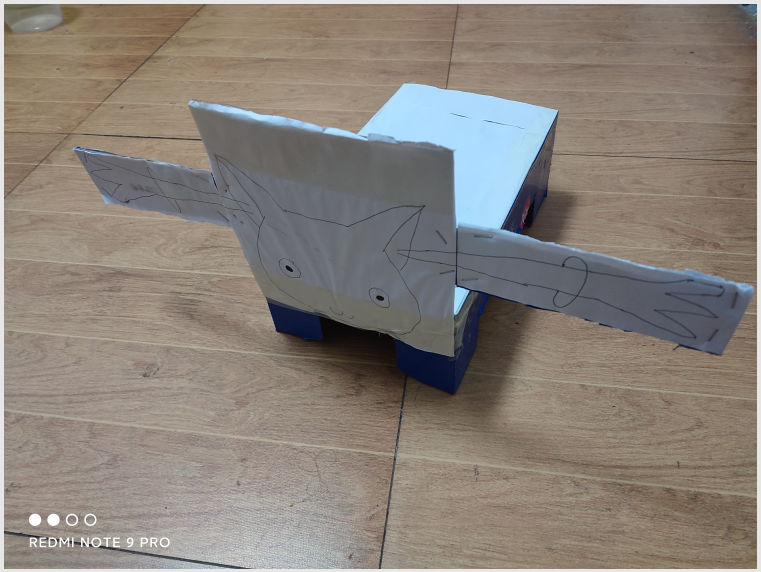
挑战级

- 理解教材练习2.63
- 裂解教材练习2.64

预告

- 目前我们每次只使用抽象数据类型的一种实现
- 在2.4节，我们将同时使用抽象数据类型的多种实现

- 使用表、树和符号解决更多问题
- 代码难度加大



疫情期间，大家要遵守政策，众志成城

下课